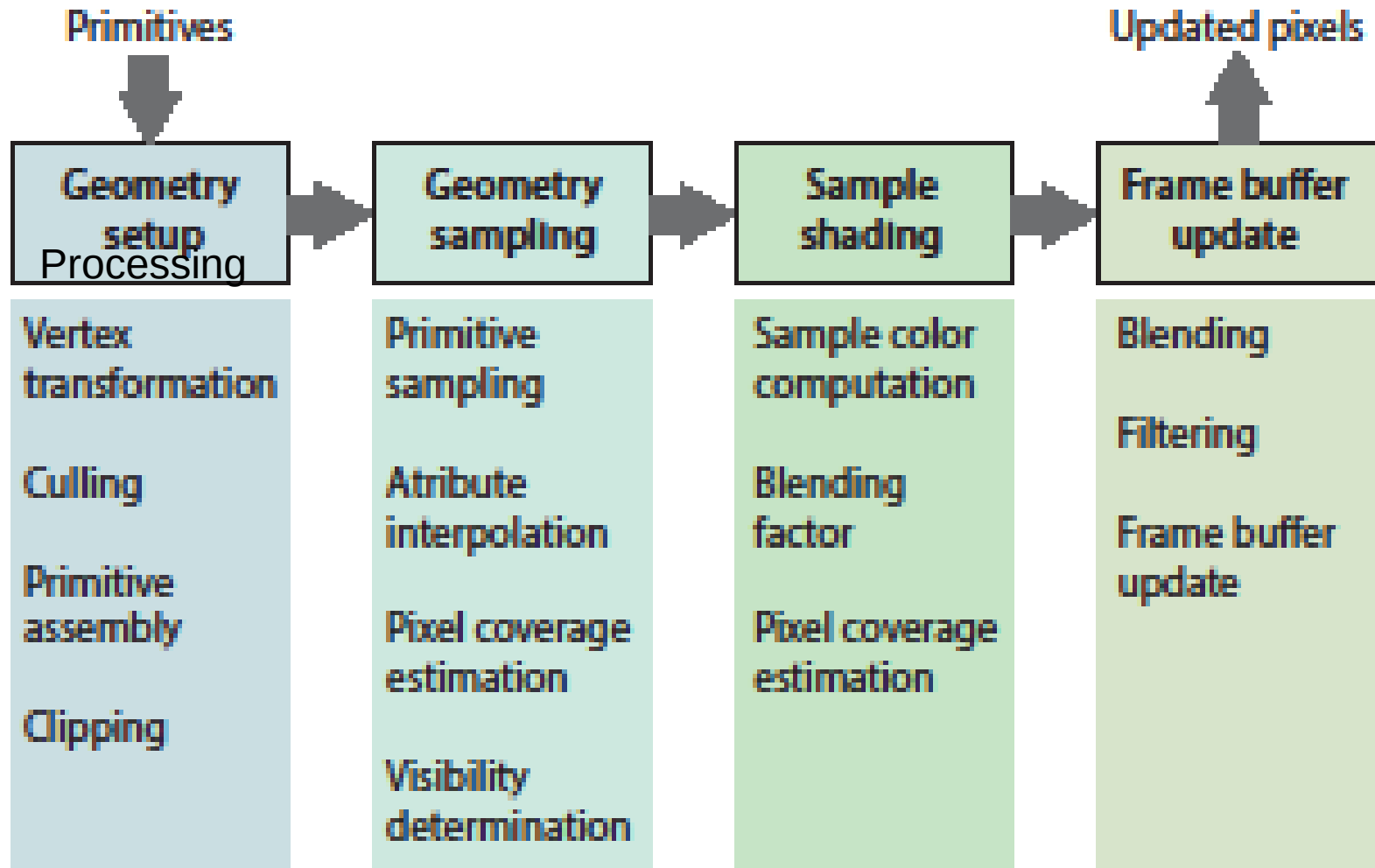


Graphics & Visualization

3D Graphics Pipeline

Graphics Pipeline: Real Operations

- What really happens on GPUs (as opposed to conceptual)

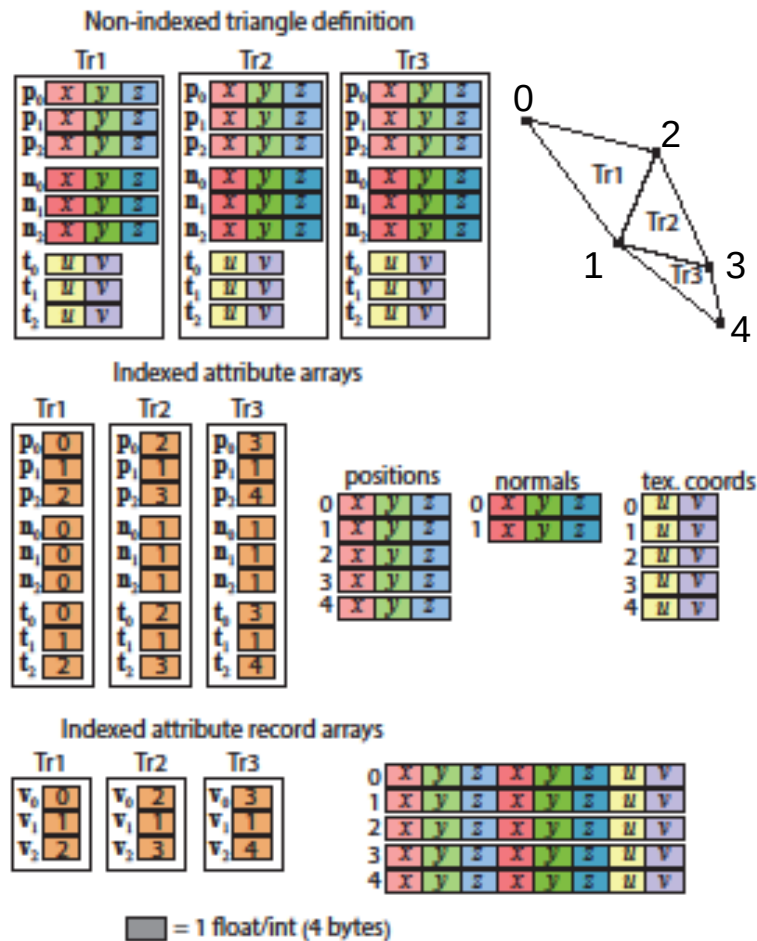


Graphics Pipeline: Real Operations (2)

- Input is primitives, usually triangles
- Output is a frame buffer image
- Primitives
 - Vertices
 - Vertex attributes
 - Connectivity information
 - Stored as Vertex Attribute Arrays
 - ◆ Connectivity information implicit or explicit
- Draw call based on pipeline state

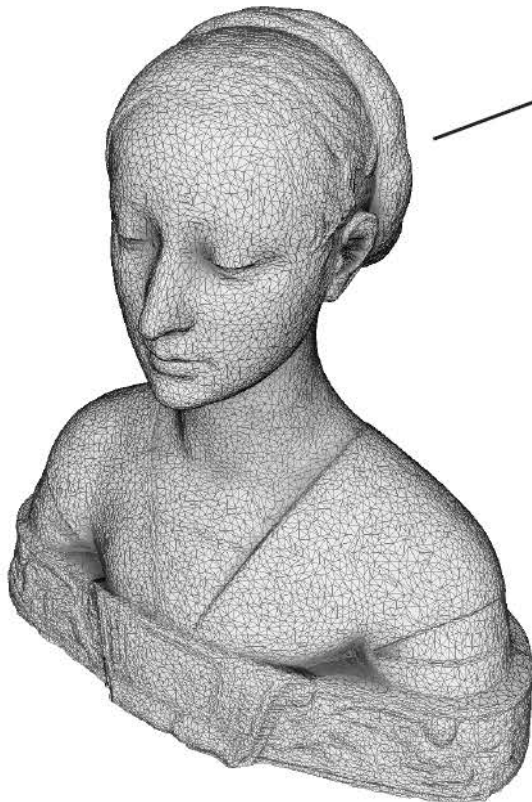
Vertex Attribute Arrays

- VAA: efficient way to store primitives sharing vertices
 - Reduce repetition by indexing



Vertex Attribute Arrays (2)

- Separate attribute arrays for [positions/normals/tex. coordinates], store *each different attribute once*; arrays can be of different length
 - Practically, different vertices/primitives seldom share attributes => extra indices take up more space than repeating *some* attributes
 - Indexed record arrays are the most common, also simple



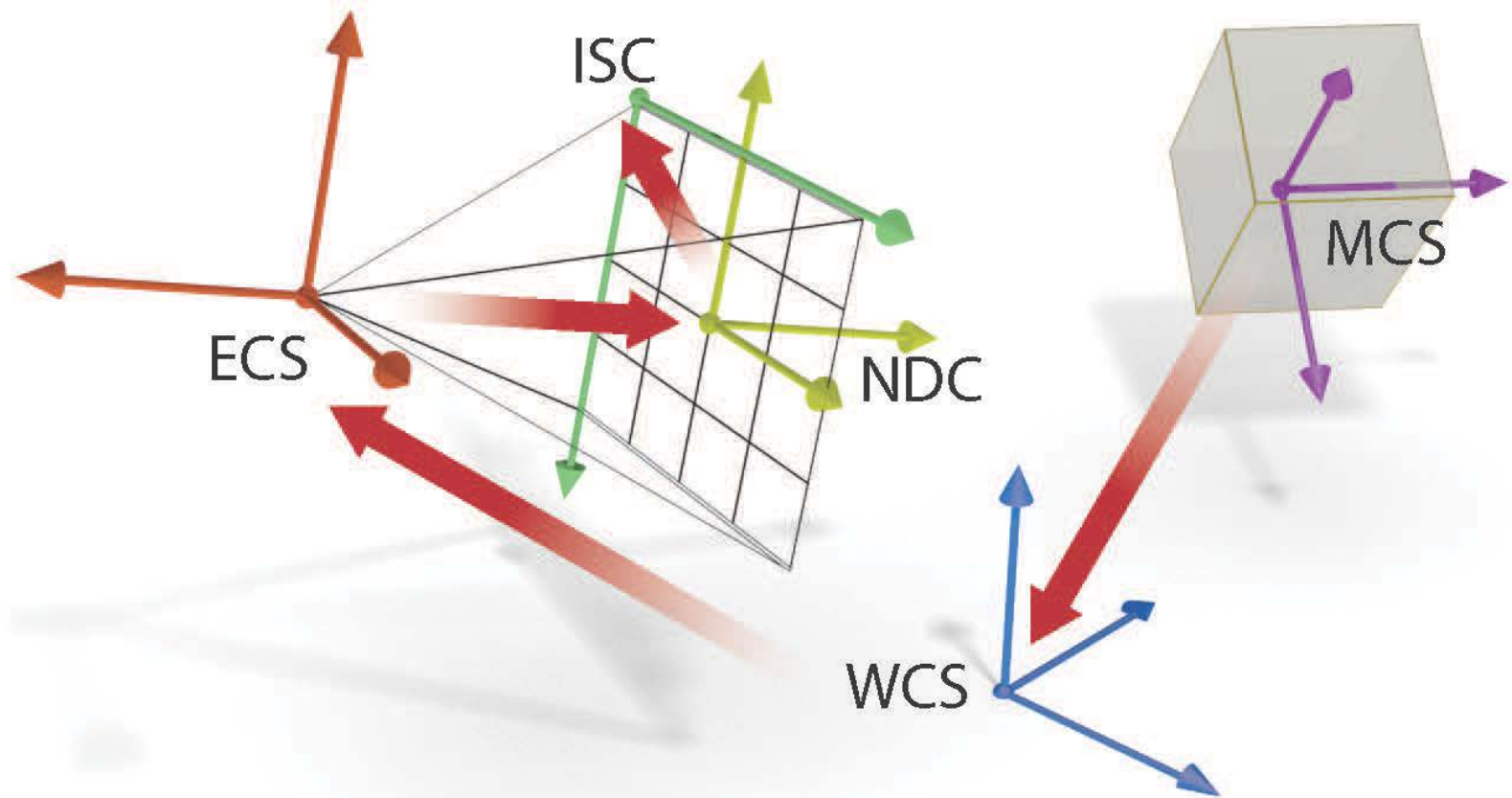
Vertex data: position + normal

Unindexed triangles: 3,596,688 bytes

Separate attribute arrays: 1,867,560 bytes

Single attribute record array: 1,268,112 bytes

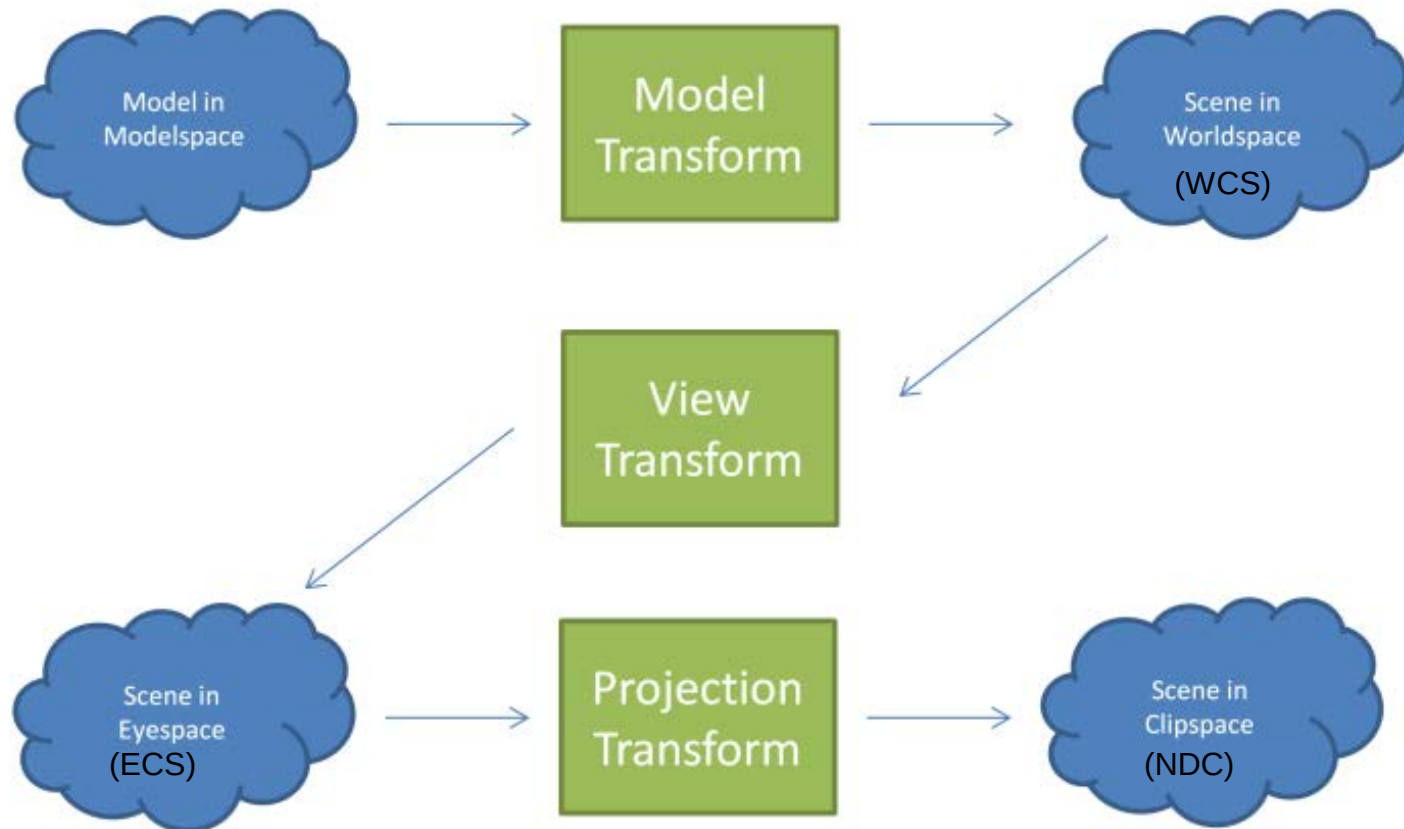
Coordinate Systems



Coordinate Systems (2)

- $\mathbf{M}_{Model}$: MCS $\rightarrow$ WCS (Scene Graph)
- $\mathbf{M}_{View}$: WCS $\rightarrow$ ECS
- $\mathbf{M}_{Projection}$: ECS $\rightarrow$ Clip Space
- Perspective Division: Clip Space $\rightarrow$ NDC
- W-2-V: NDC $\rightarrow$ ISC (translate & scale)

Coordinate Systems (3)



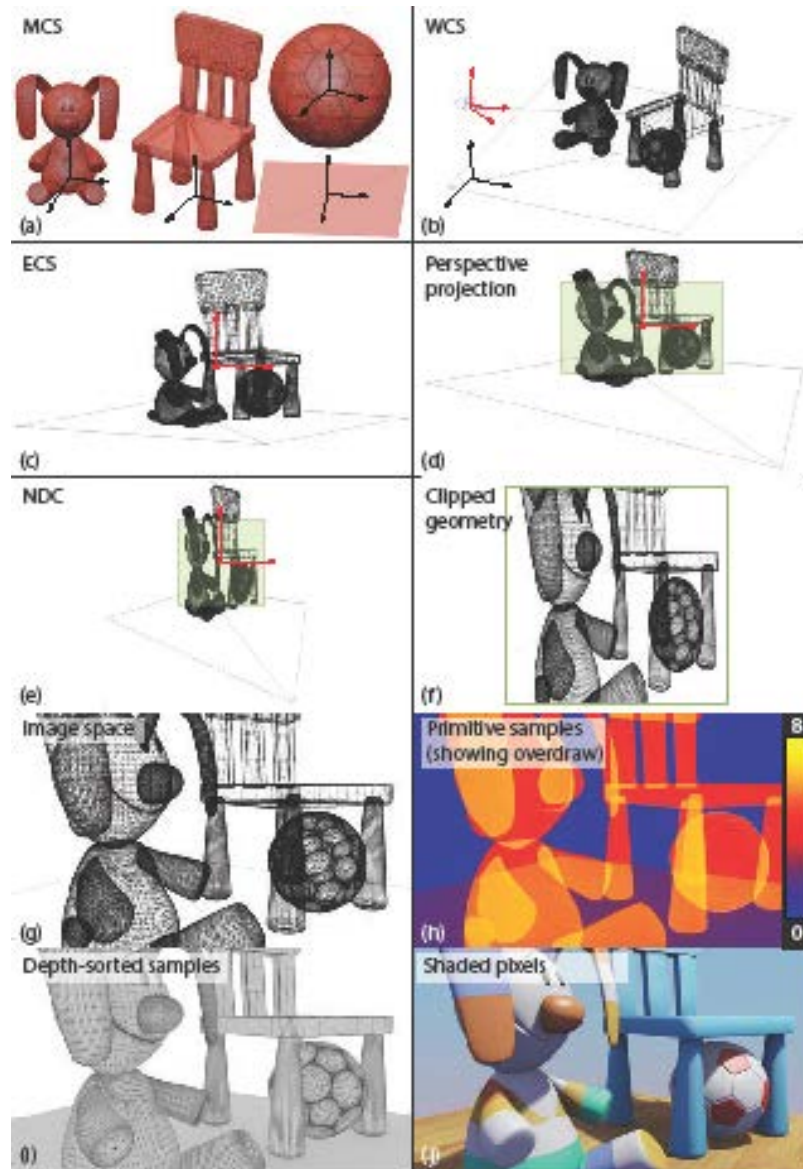
Coordinate Systems (4)



$$M_{MVP} = M_{Projection} * M_{View} * M_{Model}$$



Graphics Pipeline: Example



Geometry Processing: vertex level

- Vertices are transformed from Model Space (MCS) to Clip space:

$$\mathbf{M}_{MVP} \cdot \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} x' \\ y' \\ z' \\ z_e \end{bmatrix}$$

where $\mathbf{M}_{MVP} = \mathbf{M}_{Projection} \cdot \mathbf{M}_{View} \cdot \mathbf{M}_{Model}$

$\mathbf{M}_{Projection}$ is the $\mathbf{M}_{ECS \rightarrow NDC}^{ORTHO}$ or $\mathbf{M}_{ECS \rightarrow NDC}^{PERSP}$ matrix of Chapter 4

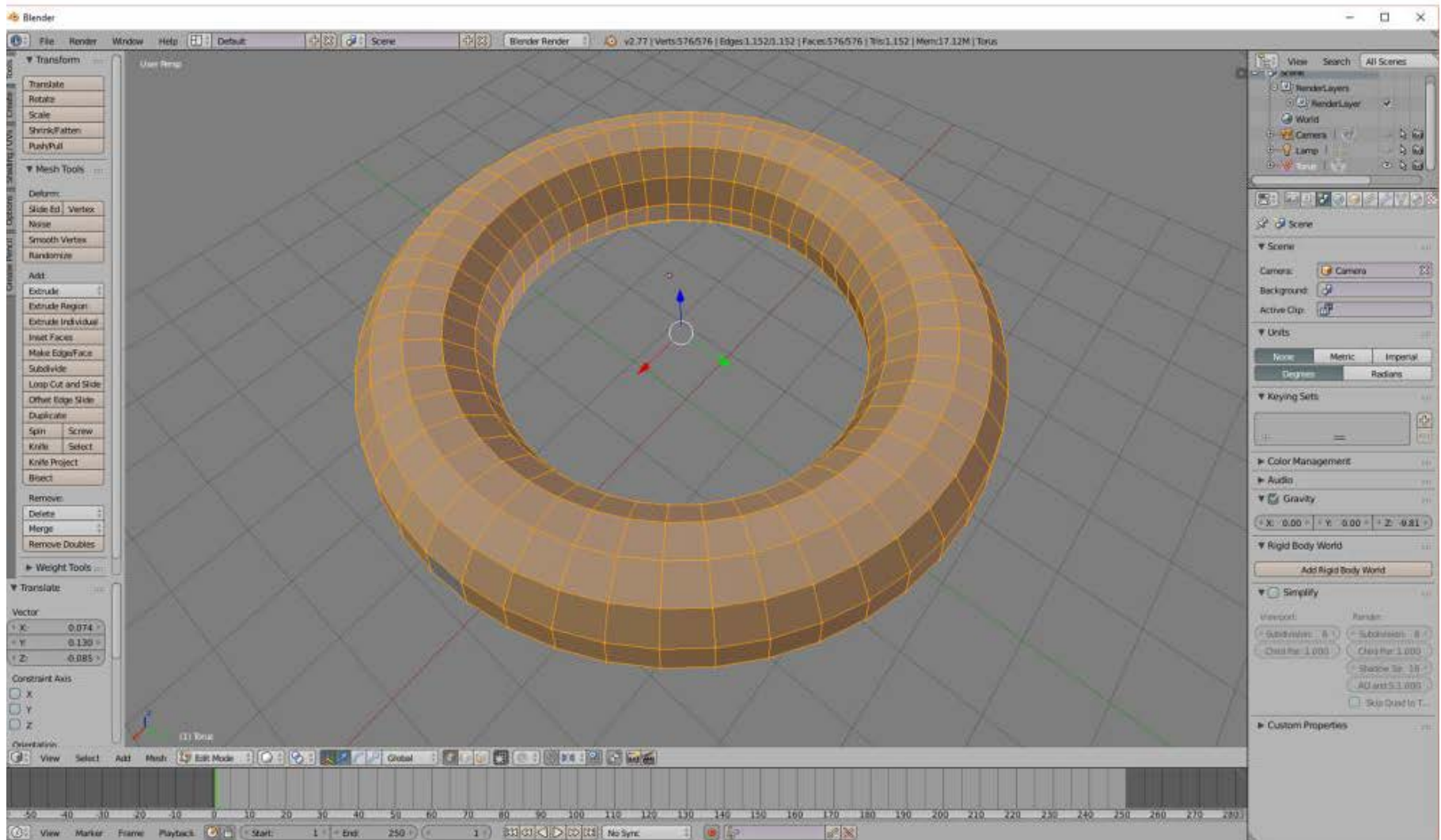
$\mathbf{M}_{View}$ is the $\mathbf{M}_{WCS \rightarrow ECS}$ matrix of Chapter 4

$\mathbf{M}_{Model}$ is the Model matrix (see below)

- Primitive culling & clipping are performed in clip space
 - Followed by *perspective division* by z_e which brings to NDC

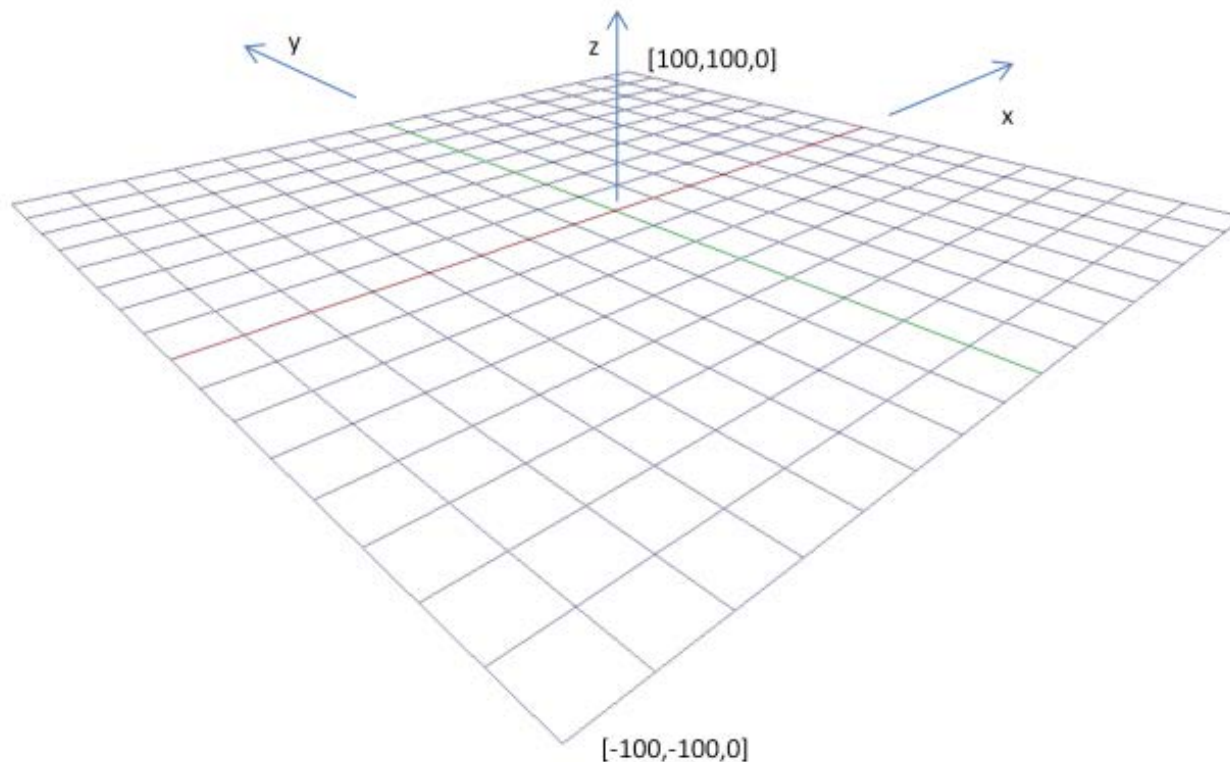
Model Transformation M_{Model}

- Start with Model Coordinate System (MCS)



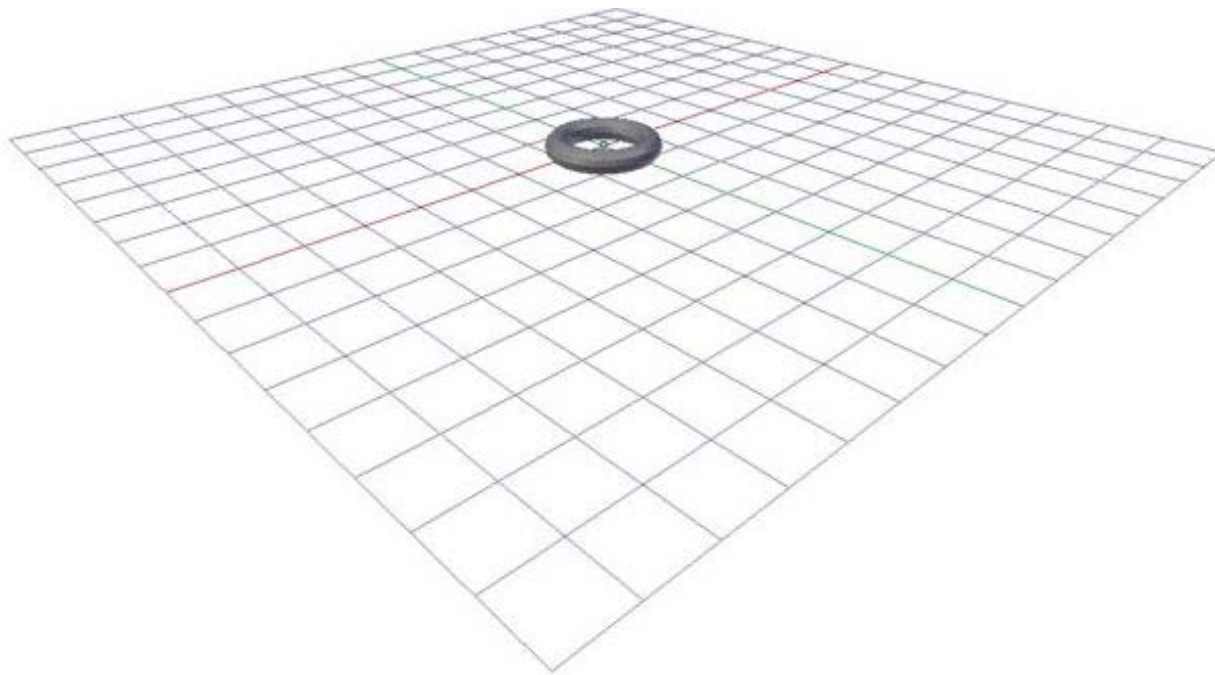
Model Transformation M_{model} (2)

- Need a common coordinate system for all models:
World Coordinate System (WCS)



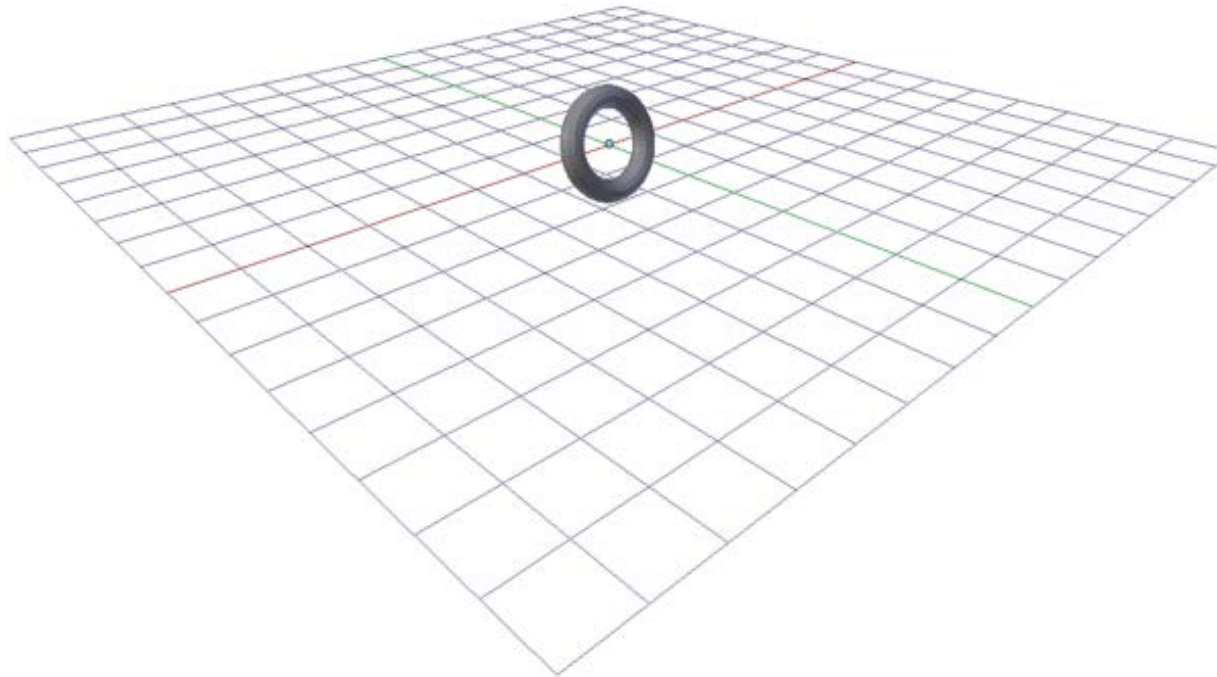
Model Transformation M_{model} (3)

- First scale model so that it takes appropriate size for WCS



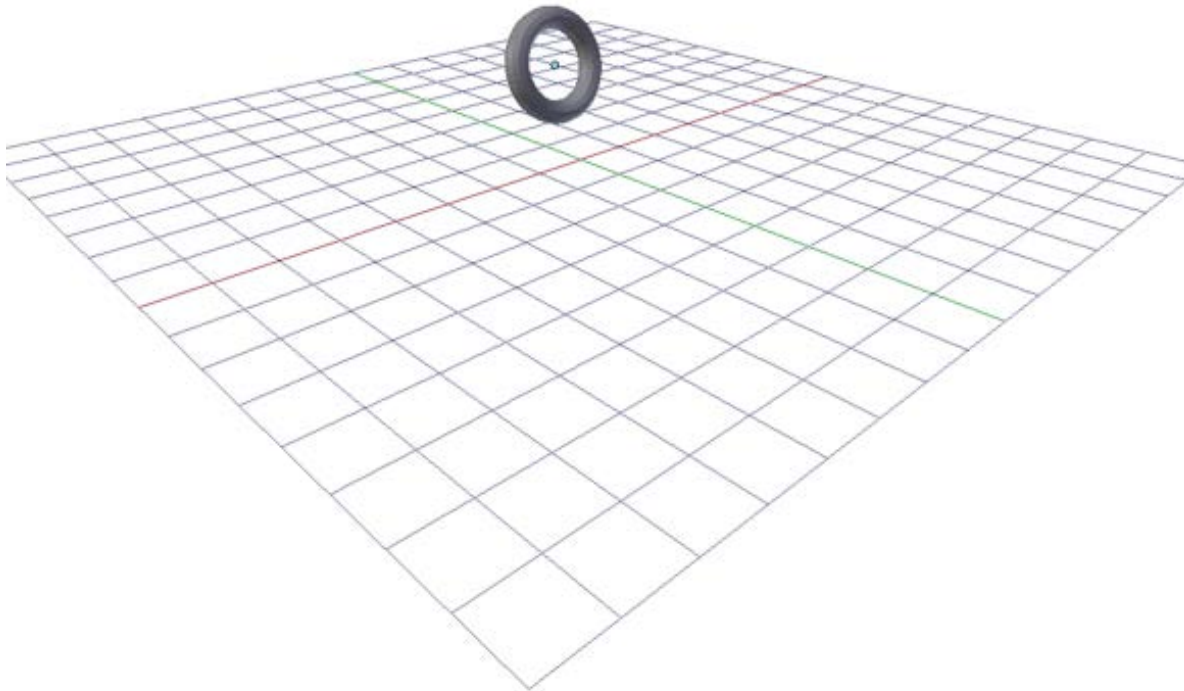
Model Transformation M_{model} (4)

- Next rotate model so that it has appropriate orientation for WCS



Model Transformation M_{model} (5)

- Finally translate model so that it is at the appropriate location in WCS



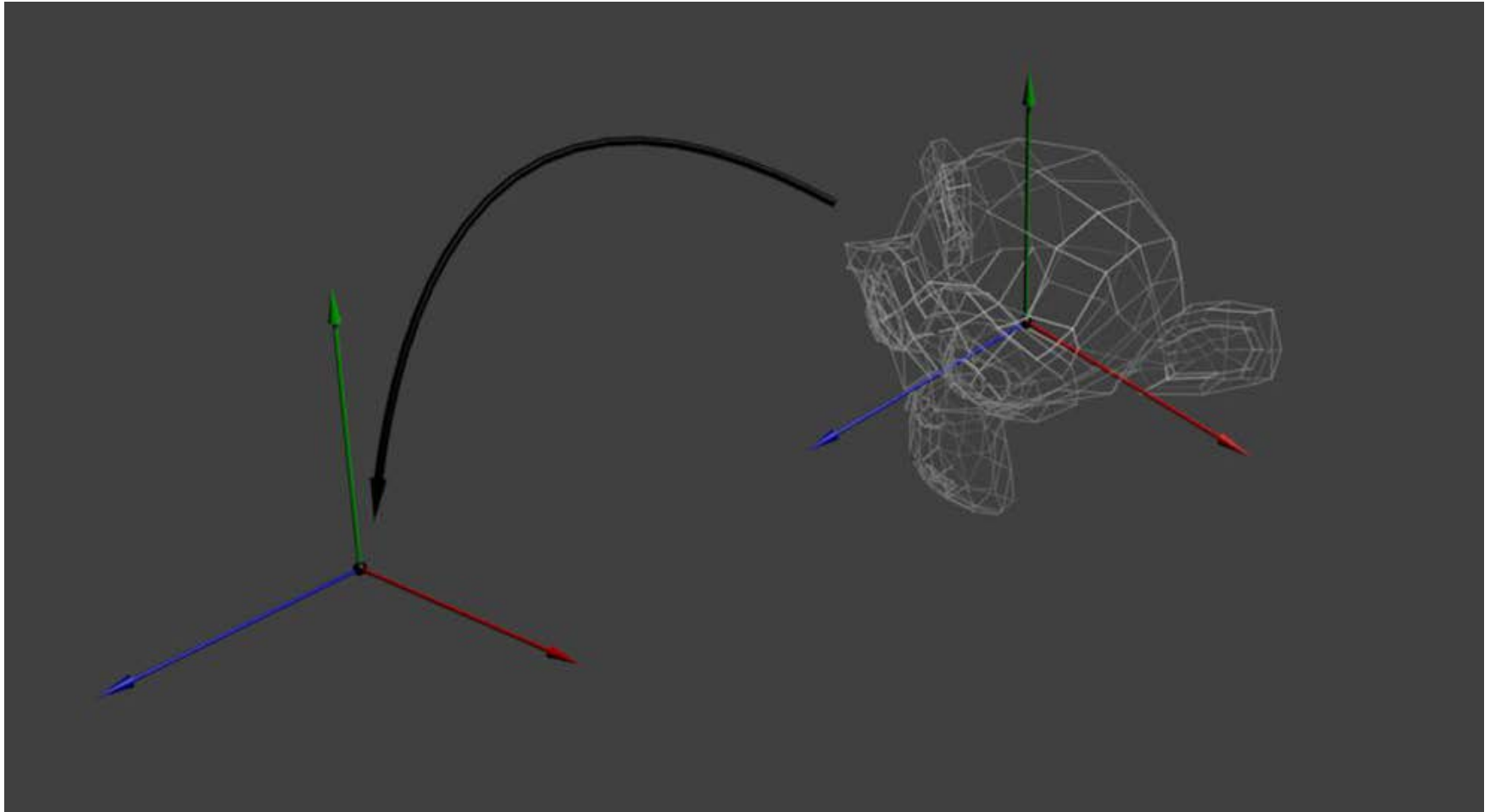
Model Transformation M_{model} (6)

- Final Matrix

$$M_{\text{Model}} = M_{\text{Translate}} * M_{\text{Rotate}} * M_{\text{Scale}}$$



Model Transformation M_{Model} (7)

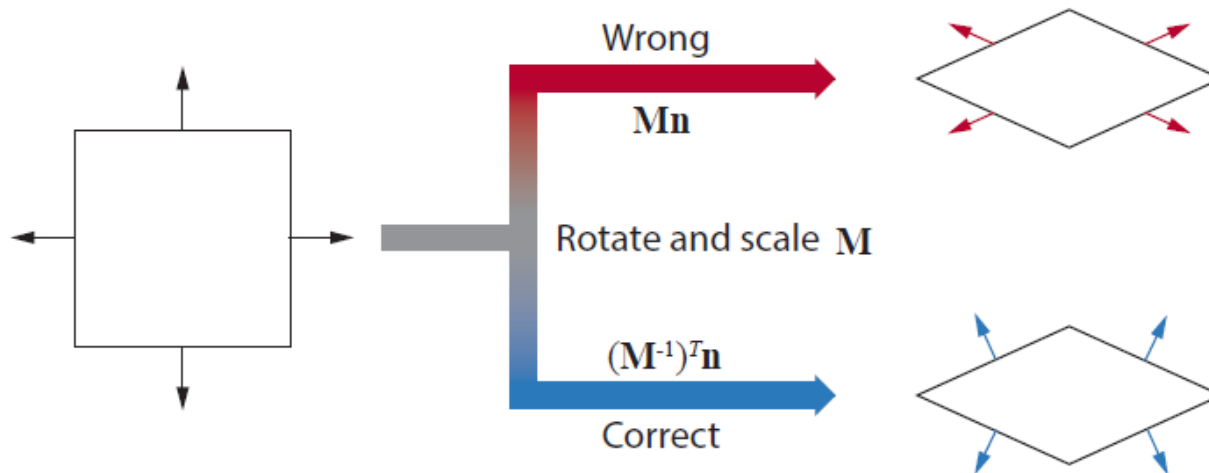


Model Transformation M_{model} (8)

- A slightly different matrix should be applied to normals:

$$M_{\text{Normal}} = (M_{\text{Model}}^{-1})^T$$

(skewing, no translation for vectors)



[proof]

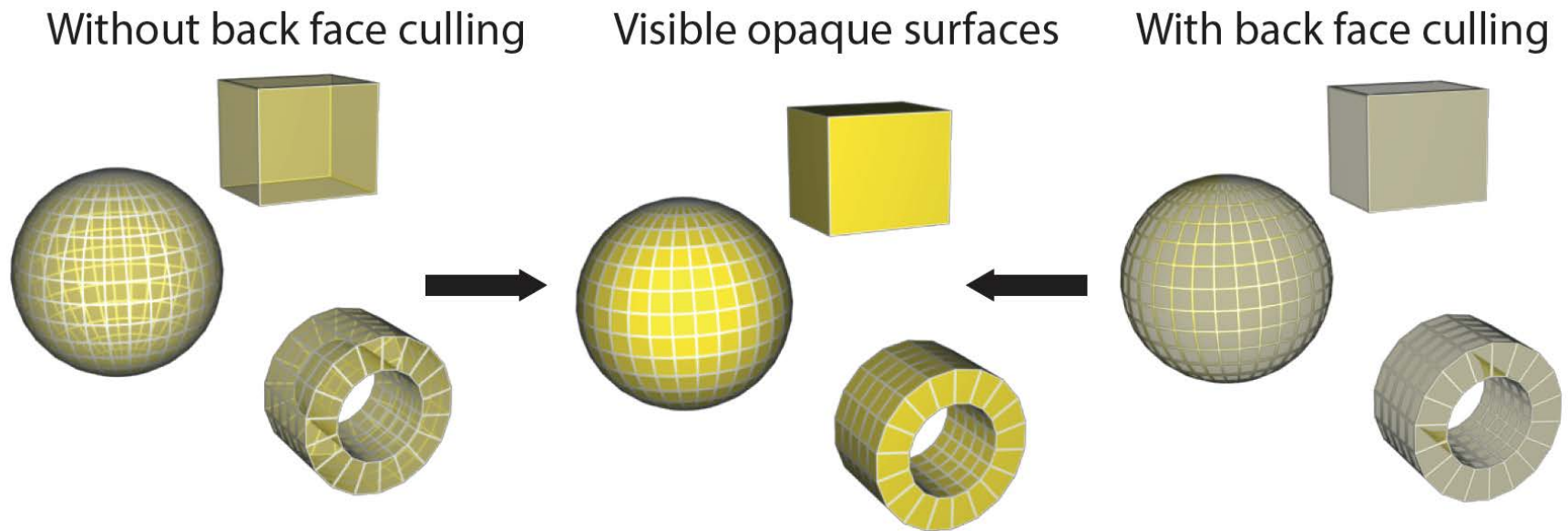
Geometry Processing: primitive level

- Example operations that require connectivity data of primitives:
 - Back-face culling (below)
 - Occlusion culling (below)
 - Frustum culling (below)
 - Frustum clipping (below)
 - Tessellation

Back-Face Culling

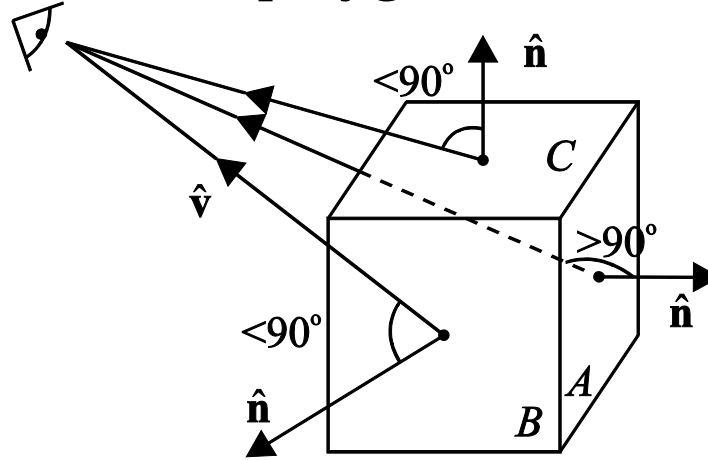
- The visible polygons of an object, are those that lie on the hemisphere facing the viewer
- If models are constructed in such a way that the back sides of polygons are never visible, then we can cull such polygons. The requirements for this are:
 - Object surfaces have no boundary (closed)
 - They are 2D manifolds
 - They are opaque
- Convexity is not a constraint
- Back faces can be detected by computing the angle formed by a polygon's normal vector $\hat{n}$ (pointing outwards from the opaque solid) and the view vector $\hat{v}$

Back-Face Culling (2)



Back-Face Culling (3)

- If angle $> 90^\circ$ then then the polygon is a back face



- The back-face test uses the inner product of the vectors :

$$\hat{\mathbf{v}} \cdot \hat{\mathbf{n}} < 0$$

- Corresponds to testing the sign of the normal, computed as cross product of 2 triangle edges, e.g. for $\mathbf{v}_0\mathbf{v}_1$ and $\mathbf{v}_0\mathbf{v}_2$, the normal:

$$\begin{vmatrix} x_1 - x_0 & x_2 - x_0 \\ y_1 - y_0 & y_2 - y_0 \end{vmatrix}$$

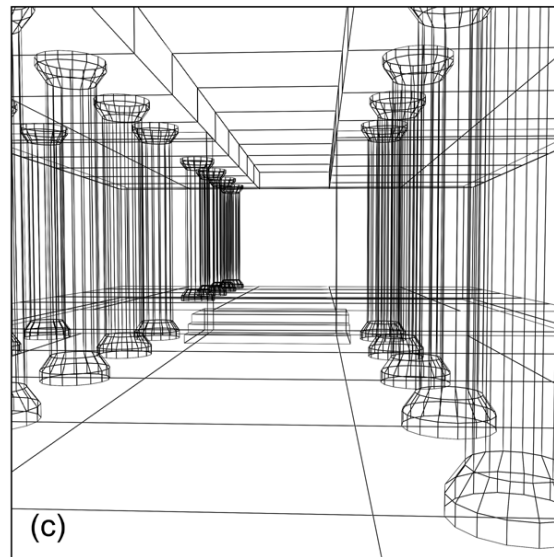
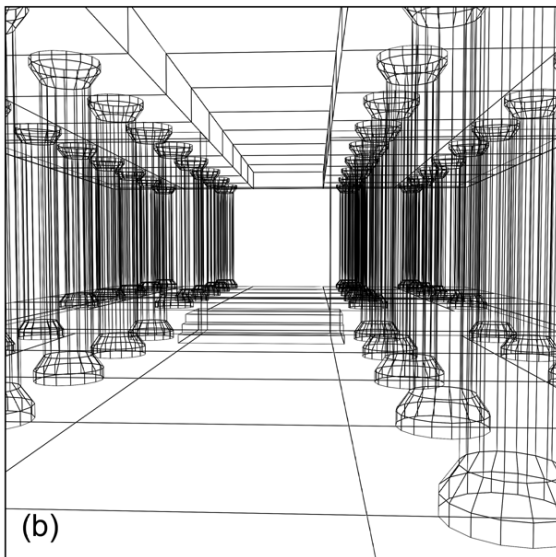
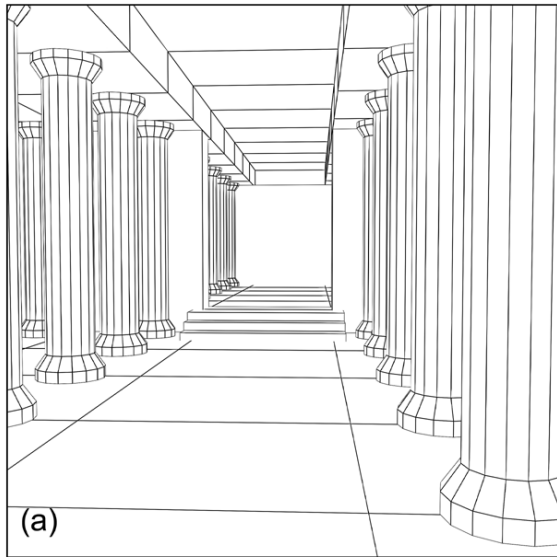
- Back face culling is extremely effective as it eliminates about 50% of the polygons

Occlusion Culling

- Occlusion culling discards primitives hidden by other primitives nearer to the observer
- Aims at efficiently discarding a large number of primitives before computationally expensive HSE algorithms are applied
- *Visible set* is the subset of primitives that are rendered on at least one pixel of the final image
- Occlusion culling algorithms compute a tight superset of the visible set so that the rest of the primitives can be discarded
- This superset is called *potentially visible set* (PVS)
- Occlusion culling does not expend time in determining *exactly* which parts of the primitives are hidden → HSE algorithms do
- Instead it determines which primitives are entirely NOT visible and quickly discards those, computing the PVS

Occlusion Culling (2)

- (a) visible set
- (b) all primitives
- (c) PVS



Occlusion Culling (3)

- PVS is then passed to HSE algorithms
- Occlusion culling costs $O(P)$, where P is # of primitives
- Two main categories of occlusion culling:
 - From-point occlusion culling:
 - ◆ Solve the occlusion problem for a single viewpoint
 - ◆ Suitable for outdoor scenes
 - From-region occlusion culling:
 - ◆ Solve the occlusion problem for an entire region of space
 - ◆ Suitable for dense indoor scenes
 - ◆ Suitable for static scenes because of the pre-computation required

From-Region Occlusion Culling

- A number of applications consists of a set of convex regions (*cells*), connected by transparent *portals*
 - Simplest form: scene represented by 2D floor plan and cells and portals are parallel to either x or y
- Primitives are only visible between cells via portals
 - Cell c_a may be visible from cell c_b via cell c_m , if appropriate sightlines exist that connect their portals
- Algorithm requires a pre-processing step
 - Its cost is only paid once assuming the cells and portals to be static
- At pre-processing, a PVS matrix and a BSP tree are constructed
- PVS matrix:
 - Gives the PVS for every cell that the viewer may be in
 - Visibility is symmetric $\rightarrow$ PVS matrix is symmetric

From-Region Occlusion Culling (2)

- It is constructed, starting from each cell c and recursively visiting all cells reachable from the cell adjacency graph, while sightlines exist that allow visibility from c
- Thus the *stab tree* of c , which defines the PVS of c , is constructed

- BSP tree:

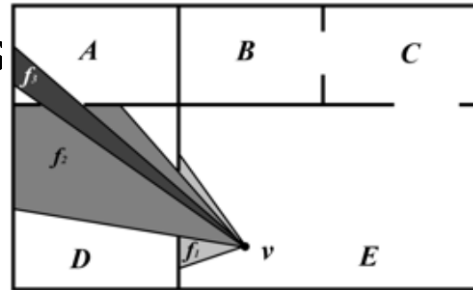
- Uses separating planes, which may be cell boundaries, to recursively partition the scene
- Leafs represent cells
- A balanced BSP tree can be used to quickly locate the cell that a point lies in, in $O(\log_2 n_c)$ time, where n_c is the # of cells

- At runtime, the steps that lead to the rendering of the PVS for a viewpoint $\mathbf{v}$ are:

- 1. Determine cell c of $\mathbf{v}$ using the BSP tree
- 2. Determine PVS of cell c using PVS matrix
- 3. Render PVS

From-Region Occlusion Culling (3)

(a) Scene modeled as cells and portals



(a)



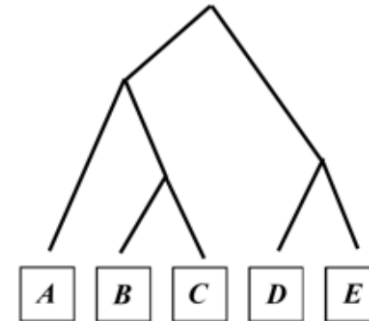
(b)

(b) Stab trees of the cells

(c) PVS matrix

	A	B	C	D	E
A	1			1	1
B		1	1		1
C		1	1	1	1
D	1		1	1	1
E	1	1	1	1	1

(c)



(d)

(d) BSP tree

- PVS does not change while $\mathbf{v}$ remains in the same cell
- The first 2 steps are only executed when $\mathbf{v}$ crosses a cell boundary
- At runtime, only the BSP tree and PVS matrix are used

From-Region Occlusion Culling (4)

- Occlusion culling can be optimized by combining it with frustum clipping and back-face culling
 - Rendering can be restricted to primitives that are both within the view frustum and the PVS
 - View frustum must be recursively constricted from cell to cell on the stab tree
- **NB: Order of culling operations does not matter!**

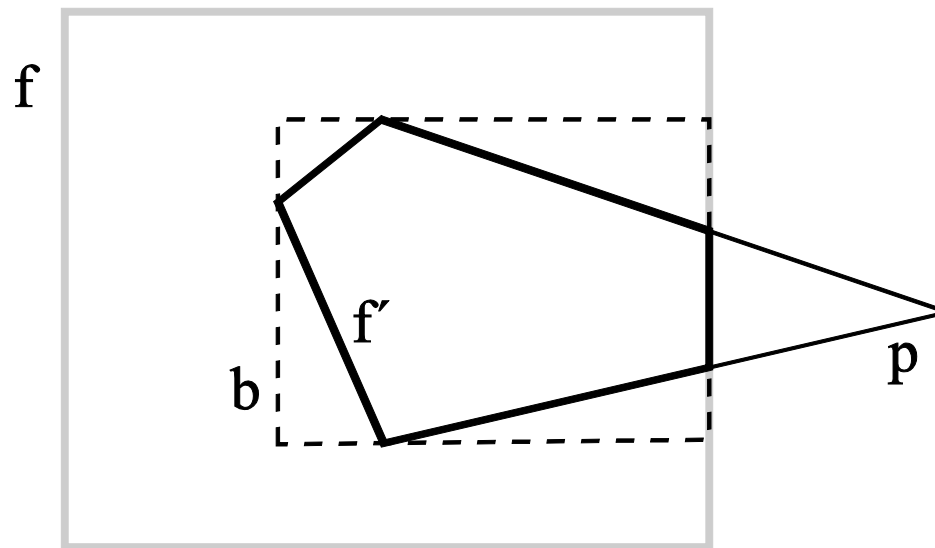
```
main()
{
    determine cell c of viewpoint using BSP tree;
    determine PVS of cell c using PVS matrix;
    f = original view frustum;
    portal_render(c, f, PVS);
}
```

From-Region Occlusion Culling (5)

```
Portal_render(cell c, frustum f, list PVS);
{
    for each polygon R in c {
        if ((R is portal) & (c' in PVS)) {
            /* portal R leads to cell c' */
            /* compute new frustum f' */
            f' = clip_frustum(f, R);
            if (f' <> empty) portal_render(c', f', PVS);
        }
        else if (R is portal) {}
        else { /* R is not portal */
            /* apply back-face cull */
            if (!back_face(R)) {
                /* apply frustum clip */
                R' = clip_poly(f, R);
                if (R' <> empty) render(R');
            }
        }
    }
}
```

From-Region Occlusion Culling (6)

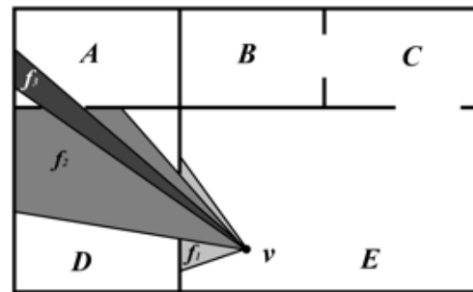
- $f' = \text{clip_frustum}(f, R)$ command computes the intersection of the current frustum f and the volume formed by the viewpoint and the portal polygon R
 - If this produces odd convex shapes, we may lose the ability to use hardware support
 - A solution is to replace f' by its bounding box



From-Region Occlusion Culling (7)

EXAMPLE:

- Cell E , that the viewer $\mathbf{v}$ lies in, is first determined
- Objects in that cell are culled against original frustum f_1
- The first portal leading to PVS cell D constricts the frustum to f_2
- Objects within cell D are culled against the new frustum
- The second portal leading to cell A reduces to frustum f_3
- Objects within cell A are culled against f_3 frustum
- Recursive process stops here as there are no new portal polygons within f_3 frustum



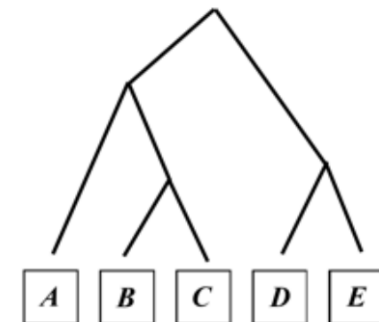
(a)



(b)

	A	B	C	D	E
A	1			1	1
B		1	1		1
C		1	1	1	1
D	1		1	1	1
E	1	1	1	1	1

(c)



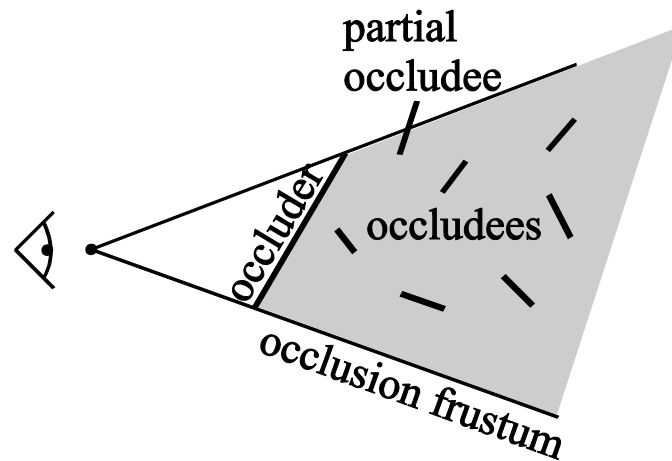
(d)

From-Point Occlusion Culling

- In outdoor scenes it cannot be assumed that the scene consists of cells and portals
- Partitioned regions of such scenes would not be coherent, with regard to their occlusion properties
- From-point occlusion culling solves the problem for a single viewpoint → does not require as much pre-processing as from-region occlusion culling, since PVS is not computed
- Main idea behind from-point techniques is the *occluder*:
 - Occluder is a primitive, or combination of primitives, that occludes a large number of other primitives (called *occludees*) with respect to a certain viewpoint
 - Region of space defined by the viewpoint and the occluder is the *occlusion frustum*

From-Point Occlusion Culling (2)

- Primitives that lie entirely within the occlusion frustum can be culled
- Partial occludees must be referred to the HSE algorithms



- Two main steps are required to perform occlusion culling for a viewpoint $\mathbf{v}$:
 - Create a small set of good occluders for $\mathbf{v}$
 - Perform occlusion culling using these occluders

From-Point Occlusion Culling (3)

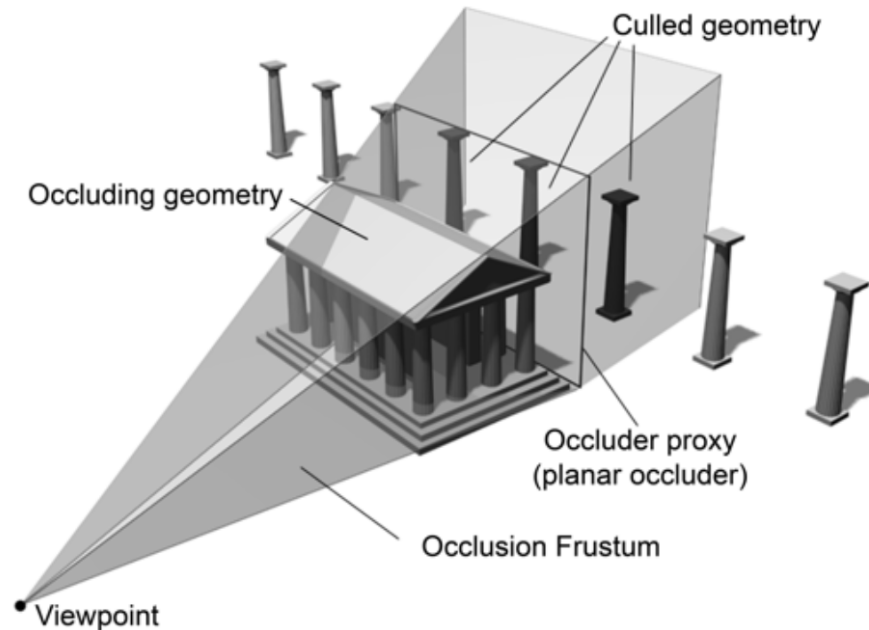
- Planar occluders ranked according to the area of screen space projections were used by Coorg and Teller
 - Their ranking function is:

$$f_{planar} = \frac{-A(\hat{\mathbf{n}} \cdot \hat{\mathbf{v}})}{|\vec{\mathbf{v}}|^2}$$

where A is the area of a planar occluder, $\hat{\mathbf{n}}$ is its unit normal vector and $\hat{\mathbf{v}}$ is the vector from the viewpoint to the center of the plane occluder

- A usual way of computing a planar occluder is as the proxy for a primitive or object
- The proxy is a convex polygon perpendicular to the view direction inscribed within the occlusion frustum of the occluder object or primitive

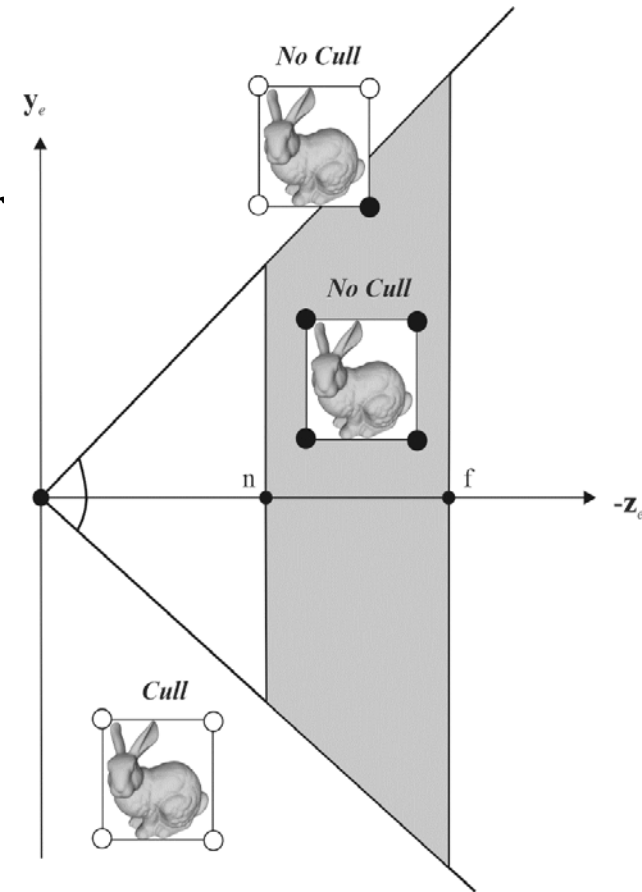
From-Point Occlusion Culling (4)



- Occlusion culling step can be made more efficient by keeping a hierarchical bounding volume description of the scene
- Starting at the top level, a bounding volume that is entirely inside/outside an occlusion frustum is rejected/rendered
- A bounding volume that is partially inside and partially outside is split into the next level of bounding volumes

Frustum Culling

- The viewing transformation defines the field of view of the observer
- The field is restricted by a minimum and maximum depth value, defining a 3D solid called *view volume* or *view frustum*
- Form of view frustum:
 - Truncated pyramid {perspective projection}
 - Rectangular parallelepiped {orthographic or parallel projection}
- Frustum culling: cull *entire* objects that fall outside view frustum
 - Implemented by checking bounding box
 - In application layer, beginning of pipeline (WCS)
 - Also known as *trivial rejection*



Frustum Clipping

- Eliminate primitives or *parts* thereof outside the view frustum
- Takes place in *clip space*
- Frustum clipping must be performed in 3D space:
 - After projection, we could not tell if objects are in front/behind observer
 - Avoid (expensive) perspective division for objects that are clipped out
 - Allow optimal bit allocation for depth (near and far clipping planes)

Frustum Clipping (2)

- Objects to be clipped:
 - Points
 - Line segments
 - polygons
- Point clipping is trivial
- Line segment and polygon clipping reduce to the computation of a line segment with the planes of the clipping object
- In 3D, the interior of the clipping object is defined as:

$$x_{min} \leq x \leq x_{max}$$

$$y_{min} \leq y \leq y_{max}$$

$$z_{min} \leq z \leq z_{max}$$

(1)

Frustum Clipping (3)

- In orthographic or parallel projection we use $\mathbf{M}_{\text{ECS} \rightarrow \text{NDC}}^{\text{ORTHO}}$ matrix which maps the clipping planes to -1 and 1 so that:

$$\begin{aligned}x_{\min} &= y_{\min} = z_{\min} = -1 \\x_{\max} &= y_{\max} = z_{\max} = 1\end{aligned}$$

- In perspective projection, the $\mathbf{M}_{\text{ECS} \rightarrow \text{NDC}}^{\text{PERSP}}$ matrix (before the division by w) maps the clipping planes to $-w$ and w so that:

$$\begin{aligned}x_{\min} &= y_{\min} = z_{\min} = -w \\x_{\max} &= y_{\max} = z_{\max} = w\end{aligned}$$

- The value of w is not constant (equal to a point's z_e)
- Clipping against w is called *homogeneous clipping*

Frustum Clipping (4)

- For a parametric line segment:

$$\mathbf{l}(t) = (1-t)\mathbf{p}_1 + t\mathbf{p}_2$$

from $\mathbf{p}_1 = [x_1, y_1, z_1, w_1]^T$ to $\mathbf{p}_2 = [x_2, y_2, z_2, w_2]^T$,

the value of w can be interpolated as :

$$(1-t)w_1 + tw_2$$

then the inequalities **(1)** can be used to define the part of the line segment within the clipping object:

$$-((1-t)w_1 + tw_2) \leq (1-t)x_1 + tx_2 \leq (1-t)w_1 + tw_2$$

$$-((1-t)w_1 + tw_2) \leq (1-t)y_1 + ty_2 \leq (1-t)w_1 + tw_2$$

$$-((1-t)w_1 + tw_2) \leq (1-t)z_1 + tz_2 \leq (1-t)w_1 + tw_2$$

(2)

Frustum Clipping (5)

- Solving the 6 inequalities for t we get the 6 intersection points of the line segment with the clipping object planes:

$$\text{left: } t = \frac{x_1 + w_1}{(x_1 - x_2) + (w_1 - w_2)}$$

$$\text{right: } t = \frac{x_1 - w_1}{(x_1 - x_2) + (w_2 - w_1)}$$

$$\text{bottom: } t = \frac{y_1 + w_1}{(y_1 - y_2) + (w_1 - w_2)}$$

$$\text{top: } t = \frac{y_1 - w_1}{(y_1 - y_2) + (w_2 - w_1)} \quad (3)$$

$$\text{near: } t = \frac{z_1 + w_1}{(z_1 - z_2) + (w_1 - w_2)}$$

$$\text{far: } t = \frac{z_1 - w_1}{(z_1 - z_2) + (w_2 - w_1)}$$

3D Clipping Algorithms

- Most 2D clipping algorithms extend easily to 3D space by addressing:
 - The intersection computation
 - The inside / outside test

3D Clipping Algorithms (2)

3D Liang – Barsky Line Clipping

- The line segment to be clipped is represented by its starting and ending points $\mathbf{p}_1$ and $\mathbf{p}_2$ (as before)
- In orthogonal or parallel projection, the clipping object is a cube and the LB computations extend directly to 3D by adding a z-coordinate inequality: $z_{min} \leq z_1 + t\Delta z \leq z_{max}$
- For perspective projection and homogeneous coordinates, we use the inequalities (2) which define the part of the a parametric line segment within the clipping object:

$$-(w_1 + t\Delta w) \leq x_1 + t\Delta x \leq w_1 + t\Delta w$$

$$-(w_1 + t\Delta w) \leq y_1 + t\Delta y \leq w_1 + t\Delta w$$

$$-(w_1 + t\Delta w) \leq z_1 + t\Delta z \leq w_1 + t\Delta w$$

$$\text{where } \Delta x = x_2 - x_1, \Delta y = y_2 - y_1, \Delta z = z_2 - z_1$$

3D Clipping Algorithms (3)

- These inequalities have the common form $tp_i \leq q_i$ for $i = 1, 2, \dots, 6$ where:

$$p_1 = -\Delta x - \Delta w \quad q_1 = x_1 + w_1$$

$$p_2 = \Delta x - \Delta w \quad q_2 = w_1 - x_1$$

$$p_3 = -\Delta y - \Delta w \quad q_3 = y_1 + w_1$$

$$p_4 = \Delta y - \Delta w \quad q_4 = w_1 - y_1$$

$$p_5 = -\Delta z - \Delta w \quad q_5 = z_1 + w_1$$

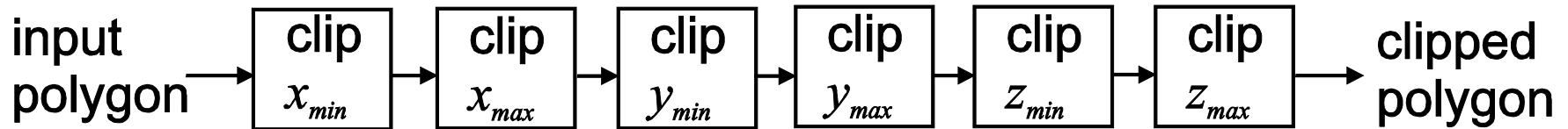
$$p_6 = \Delta z - \Delta w \quad q_6 = w_1 - z_1.$$

- Notice that the ratios (q_i / p_i) correspond to the parametric intersection values of the line segment with clipping plane i
- The rest of the LB algorithm remains as in 2D

3D Clipping Algorithms (4)

3D Sutherland – Hodgman Polygon Clipping

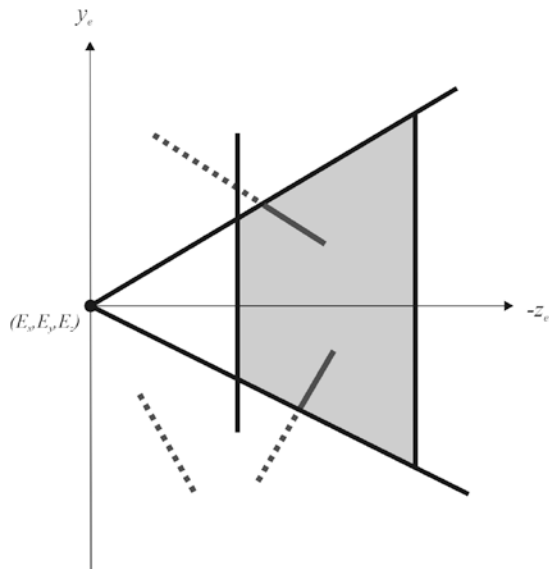
- In 3D the clipping object is a convex volume, the view frustum, instead of a convex polygon
- The algorithm consists of 6 pipelined stages, one for each face of the view frustum



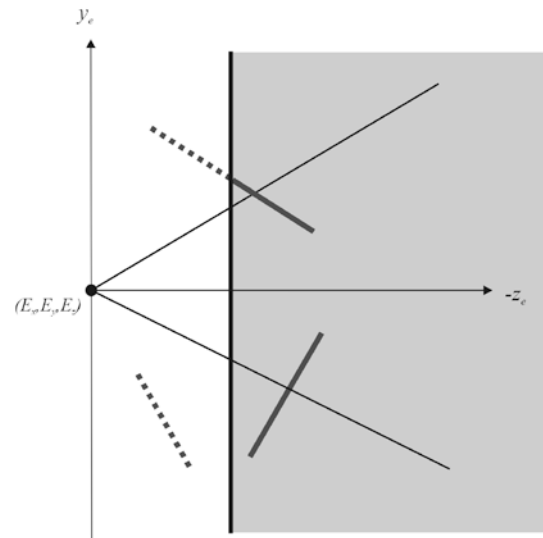
- Main differences against the 2D algorithm are:
 - The `inside_test` subroutine must be altered so that it tests whether a point is on the inside half-space of a plane → equivalent to testing the sign of the plane equation for the coordinates of the point
 - The `intersect_lines` subroutine must be replaced by `intersect_plane_line` to compute the intersection of a polygon edge against a plane of the clipping volume → equations (3) are used for perspective projection

1-plane Clipping

- Efficient alternative to frustum clipping:
 - 1. Frustum culling
 - 2. Clipping against $z_e = \text{near}$ only



Full Clip



1-plane Clip

- Divisions by 0 and objects behind observer avoided
- Reduces degenerate cases

Geometry Sampling

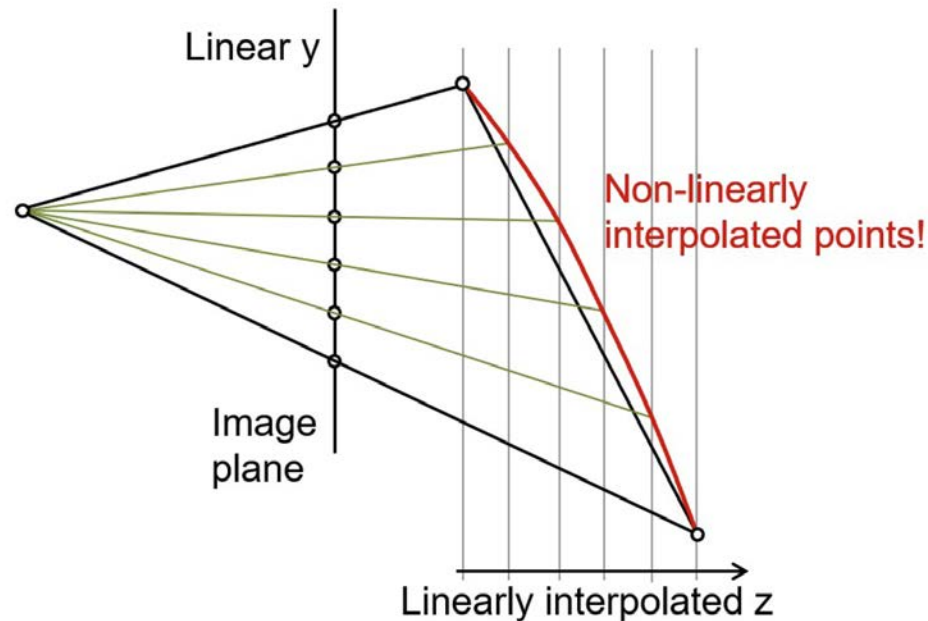
- Geometry sampling: rasterization of geometric primitives (triangles, line segments etc) by sampling them on the pixel grid
 - => Primitive Rasterization algorithms (Rasterization chapter)
- During sampling:
 - Attributes need to be interpolated from vertices (below)
 - Hidden parts need to be determined (below)

Attribute Interpolation: Perspective Correction

- The rasterization of primitives is performed in 2D screen space (NDC/ISC) while the properties of primitives are associated with 3D object vertices (MCS)
- Perspective projection does not preserve ratios of distances → incorrect to linearly interpolate the attribute values of vertices in screen space
- => **Perspective Correction**
 - Based on the fact that perspective projection preserves cross ratios

Attribute Interpolation: Perspective Correction (2)

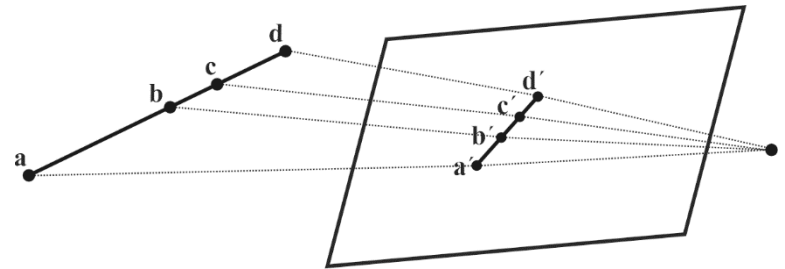
Linear in screen space $\rightarrow$ Non-linear in eye space!



Attribute Interpolation: Perspective Correction (3)

- On a line segment:
 - Let **ad** be a line segment and **b** its midpoint in 3D space
 - Let **a', d', b'** be the perspective projections of the points **a, d, b**

$$\left. \begin{array}{l} \frac{ac}{cd} = \frac{a'c'}{c'd'} \\ \frac{ab}{bd} = 1, \quad \frac{a'b'}{b'd'} = q \end{array} \right\} \frac{ac}{cd} = \frac{a'c'}{qc'd'}$$



- Heckbert provides an efficient solution (perspective correction):
 - Attributes essentially go through perspective division like position coordinates
 - Let $[x, y, z, c, w]^T$ be the pre-perspective coordinates of a vertex (attribute c)
 - Keep $[x/w, y/w, z/w, c/w, 1/w]^T$ for the projected vertex (note $w=z$)
 - Interpolate (in screen space) attribute c/w and $1/w$ value from vertices
 - ‘Correct’ attributes by division by interpolated $1/w$

Attribute Interpolation: Perspective Correction (4)

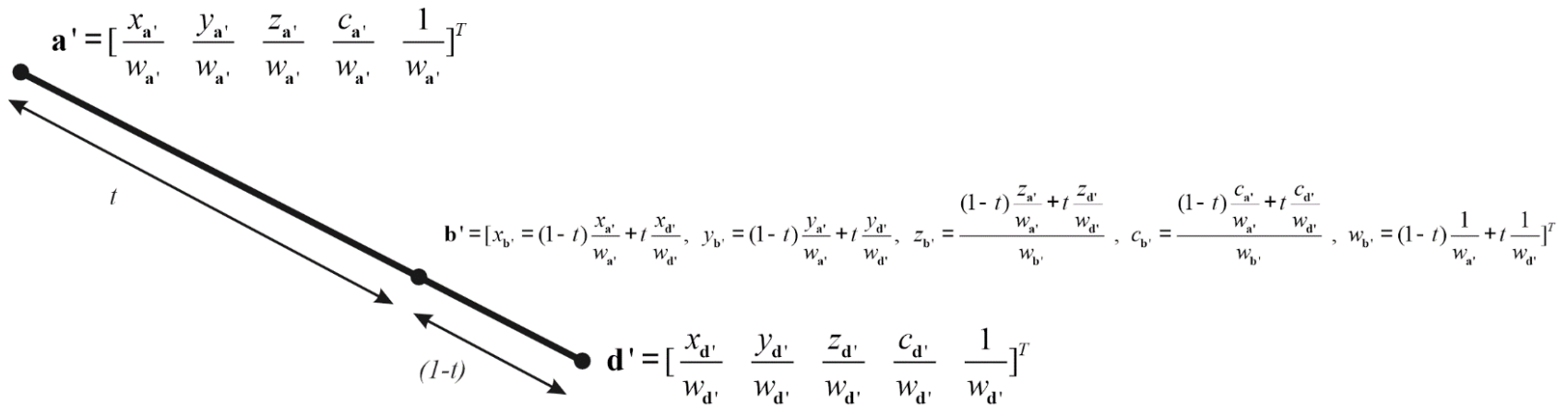
- Example 1: line segment point from NDC endpoints

$$\mathbf{a}' = \left[\frac{x_{a'}}{w_{a'}}, \frac{y_{a'}}{w_{a'}}, \frac{z_{a'}}{w_{a'}}, \frac{c_{a'}}{w_{a'}}, \frac{1}{w_{a'}} \right]^T \text{ and } \mathbf{d}' = \left[\frac{x_{d'}}{w_{d'}}, \frac{y_{d'}}{w_{d'}}, \frac{z_{d'}}{w_{d'}}, \frac{c_{d'}}{w_{d'}}, \frac{1}{w_{d'}} \right]^T$$

define internal point $\mathbf{b}'$ using normalized ratio $t = \frac{\mathbf{a}' \mathbf{b}'}{\mathbf{a}' \mathbf{d}'}$

then t and $(1-t)$ are the weights of the endpoints for $\mathbf{b}'$ and

value of the attribute c at $\mathbf{b}'$ is:

$$c_{b'} = \frac{(1-t) \frac{c_{a'}}{w_{a'}} + t \frac{c_{d'}}{w_{d'}}}{(1-t) \frac{1}{w_{a'}} + t \frac{1}{w_{d'}}}$$


Attribute Interpolation: Perspective Correction (5)

- Example 2: internal triangle point $\mathbf{q}$ with barycentric coordinates b_0, b_1, b_2 wrt NDC vertices $\mathbf{p}_0, \mathbf{p}_1, \mathbf{p}_2$:

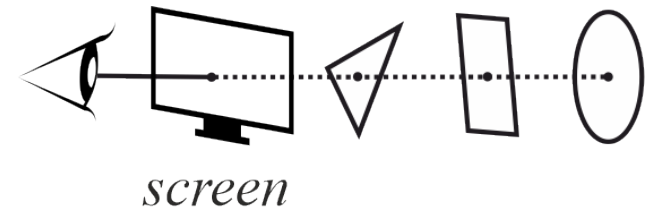
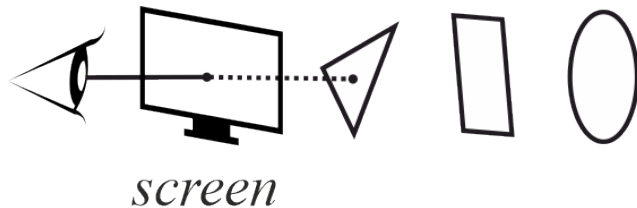
$$\mathbf{p}_i = \left[\frac{x_i}{w_i}, \frac{y_i}{w_i}, \frac{z_i}{w_i}, \frac{c_i}{w_i}, \frac{1}{w_i} \right]^T$$

the value of the attribute c at $\mathbf{q}$ is:

$$c_q = \frac{b_0 \frac{c_0}{w_0} + b_1 \frac{c_1}{w_1} + b_2 \frac{c_2}{w_2}}{b_0 \frac{1}{w_0} + b_1 \frac{1}{w_1} + b_2 \frac{1}{w_2}}$$

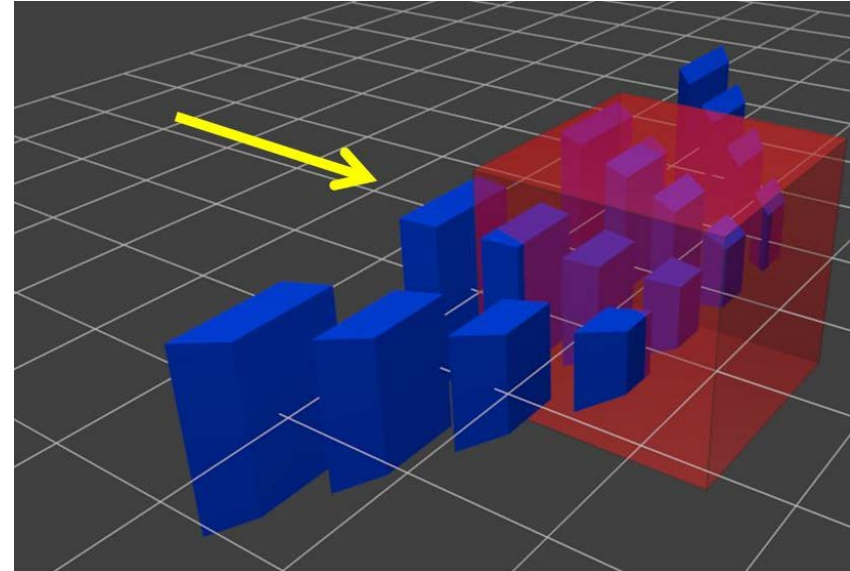
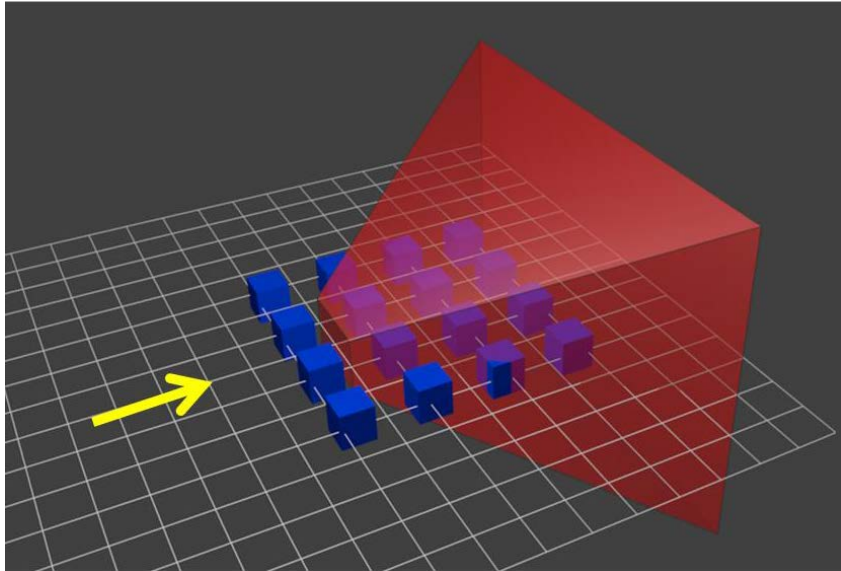
Hidden Surface Elimination

- The occlusion problem:
 - *Determination of the visible object in every part of the image*
 - Can be solved by computing the *first* object intersected by each relevant ray emanating from the viewpoint
 - Corresponds to the *minimum finding* problem, $O(n)$
 - If transparency is assumed, then we have an $O(n \log n)$ *sorting* problem
 - ◆ Only objects with x,y overlap need be sorted



Hidden Surface Elimination (2)

- Perspective projection: for simpler determination of x,y overlap, perform HSE *after* perspective division in NDC / ISC

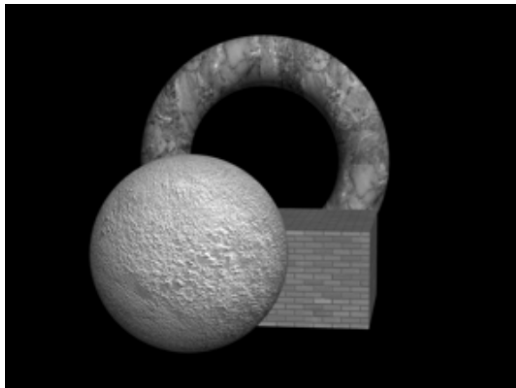


- The costly perspective division is performed anyway in the ECS to NDC perspective projection
 - => maintain z coordinate

Z – Buffer Algorithm

- Elegant and simple HSE algorithm
- Minimum finding $\Rightarrow$ 2D array of minimum depths (Z-buffer)
 - same spatial resolution as the frame buffer (1:1 x,y correspondence)
 - available on every GPU
 - can process primitives in *arbitrary* order

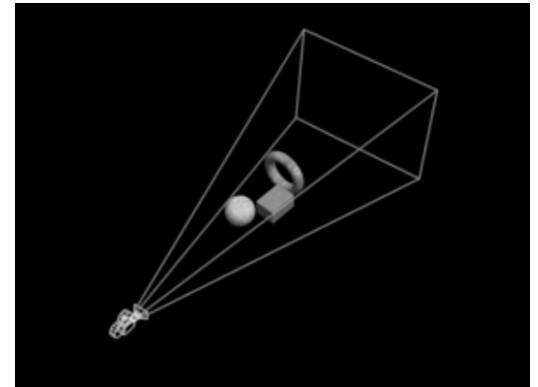
frame – buffer



Z – buffer



3D scene



Z – Buffer Algorithm (2)

- Z-buffer is initialized to a maximum (far clipping plane)
- For each primitive, during rasterization, interpolate attributes (z_p, c_p) per pixel $\mathbf{p} = (x_p, y_p)$
 - z_p is the depth of the primitive at $\mathbf{p}$ and c_p its color at $\mathbf{p}$
- Assuming that depth values decrease away from the view point, the heart of the Z-buffer is the *depth test*:

```
if (z-buffer[xp, yp] < zp) {  
    f-buffer[xp, yp] = cp; /* update frame buffer */  
    z-buffer[xp, yp] = zp; /* update depth buffer */  
}
```

Z – Buffer Algorithm (3)

- Efficient computation of the attributes (z_p, c_p) per covered pixel is critical:
 - Take advantage of surface coherence and forward differences
 - ◆ Let the *plane* equation of the primitive be:
$$F(x, y, z) = ax + by + cz + d = 0$$
 - ◆ Solving for the depth (z) :
$$F'(x, y) = z = -d/c - (a/c)x - (b/c)y$$
 - ◆ F' is incrementally computed from pixel (x, y) to pixel $(x+1, y)$:
$$F'(x+1, y) - F'(x, y) = -a/c$$
 - ◆ In practice, depth values at vertices of the planar primitive are interpolated across its edges and then across the scanlines (1+ / pixel)
 - If the attributes are only required for a single pixel, as e.g. in ray tracing, then barycentric coordinates are used instead of interpolation

Z – Buffer Algorithm (4)

- Complexity of Z-buffer algorithm is $O(Ps)$, where P # of primitives and s average # of pixels covered by a primitive
- Alternatively, $O(pD)$, where p the number of pixels and D the *depth complexity*
 - D represents average number of depth tests per pixel
- In practice, as P increases, s decreases proportionally → constant D (constant performance)
- Advantages of the algorithm:
 - Simplicity
 - Arbitrary order
 - Generality (primitives)
 - ~Constant performance
- Weaknesses of the algorithm:
 - Difficulty to handle some special effects (transparency)

Z – Buffer Algorithm (5)

- Z-buffer computed during a rendering, can be kept and used in various ways:
 - Allows depth merging of 2 or more images. Suppose (F_a, Z_a) and (F_b, Z_b) represent the frame- and Z-buffers of 2 parts of a scene. These can be merged by selecting the part with the nearest depth value at each pixel:

```
for (x=0; x<XRES; x++)  
    for (y=0; y<YRES; y++) {  
        Fc[x, y] = (Za[x, y]>Zb[x, y])? Fa[x, y]:Fb[x, y];  
        Zc[x, y] = (Za[x, y]>Zb[x, y])? Za[x, y]:Zb[x, y];  
    }
```

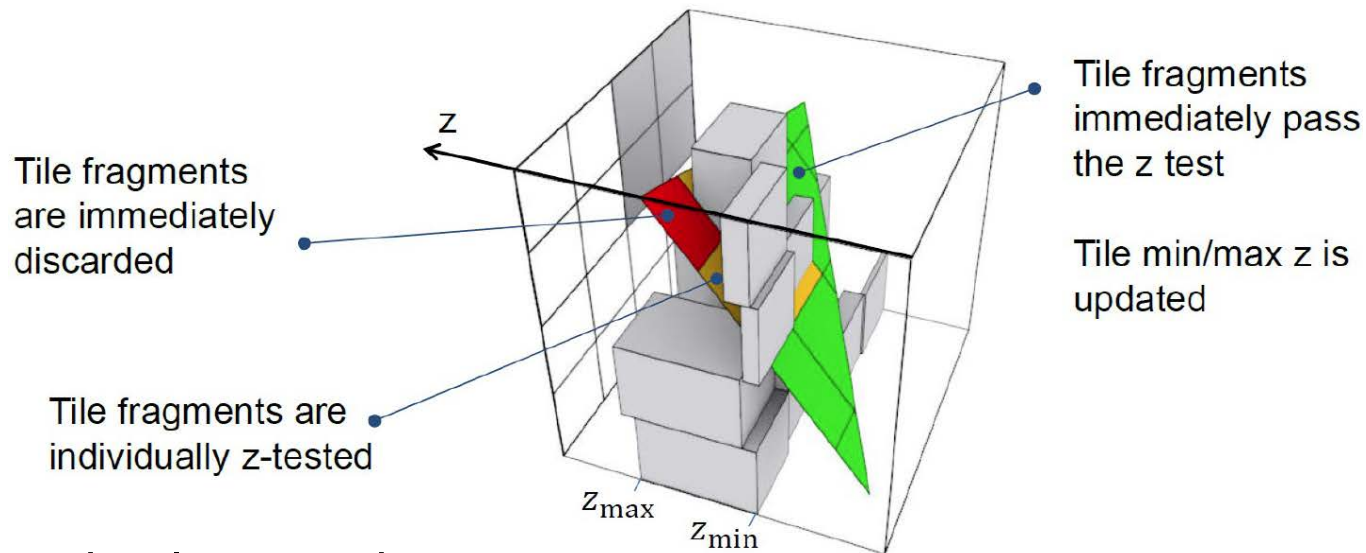
Z – Buffer Algorithm (6)

- Many more computations can be performed using Z-buffer:
 - Shadow determination
 - Voxelization
 - Voronoi computation
 - Object reconstruction
 - Symmetry detection
 - Object retrieval

Hierarchical Z-Buffer

- Hi-Z stores power-of-2 reductions of Z-Buffer
 - Each pixel in a reduction stores *min/max* depth of next higher resolution

if (primitive's *depth* < *min*) $\Rightarrow$ visible
else if (primitive's *depth* > *max*) $\Rightarrow$ not visible
else higher resolution must be checked



- GPU implementations
- Useful in culling also

Sample Shading

- The samples (at pixel level) have to be shaded (coloured)
- During shading:
 - Pixel coverage needs to be estimated (Antialiasing in Rasterization chapter)
 - Illumination must be estimated (Illumination chapter)
 - Transparency must be estimated (below)

Transparency

- Requires primitive sorting and colour blending, aka *alpha compositing*
 - Sort back-to-front and use of *over* operator
$$C_{pixel} = C_{prim} * \alpha_{prim} + C_{pixel} * (1 - \alpha_{prim})$$
 - Sort front-to-back and use of *under* operator
$$C_{pixel} = (C_{prim} * \alpha_{prim}) * \alpha_{pixel} + C_{pixel}$$
$$\alpha_{pixel} = (1 - \alpha_{prim}) * \alpha_{pixel}$$
- Where $C_{pixel} / 0 \leq \alpha_{pixel} \leq 1$: pixel colour/opacity, $\alpha_{pixel} = 1$ initially (opaque)
 $C_{prim} / 0 \leq \alpha_{prim} \leq 1$: primitive colour/opacity

Transparency (2)

- A-buffer antialiasing can be extended to transparent objects
 - A-buffer already assumes front-to-back primitive sorting
 - Replace $M_{SCH}, M_{POL}, M_{VIS}$ (boolean) by $R_{SCH}, R_{POL}, R_{VIS}$ (real)
 - $a_{pixel} a_{prim}$ correspond to $R_{SCH} R_{POL}$
 - Initialize R_{SCH} to 1.0
 - Polygon's visibility and next value of search matrix are then:
$$R_{VIS} = R_{SCH} * R_{POL}$$
$$R'_{SCH} = R_{SCH} * (1.0 - R_{POL})$$
 - Polygon's colour weight A_i is normalized sum of R_{VIS} elements:
$$A_i = \sum_j R_{VIS,j} / 32$$
 - Antialiasing, hidden surfaces and transparency are thus handled at subpixel level
 - Problems:
 - ◆ Textures
 - ◆ Primitive ordering => *Order Independent Transparency* techniques